Unit.4

Page No.
Date.

Title : Full Binary Tree Validation

Problem Statement:
To write a C++ program to determine whether a given binary tree is a full binary tree or not.
A full binary tree is a binary tree in which every node has either zero children or exactly two children. The program checks each node of the tree and displays whethers the given tree satisfies properties of a full binary tree.

Objectives :
- To understand concept of properties of a binary tree.
- To study the structure and implementation of a binary tree using C++.
- To understand concept of a full binary tree.
- To create binary tree nodes using a class and pointers.
- To check whether each node has either zero or two children.
- To implement a recursive function to validate a full binary tree.

Algorithm:

Step 1: Start the program.

Step 2: Define a Node class containing data & pointers to the left & right children.

Step 3: Create the binary tree by allocating memory for each node.

Step 4: Define a function named isFullBinaryTree() to validate the tree.

Step 5: Check whether the current node is NULL. If it is NULL, return tree.

Step 6: Check whether both left & right children are NULL. If both are NULL, node is a leaf node, so return true.

Step 7: Check whether both left & right children exist. If both exist, recursively check left & right subtrees.

Step 8: If current node has only one child, return false.

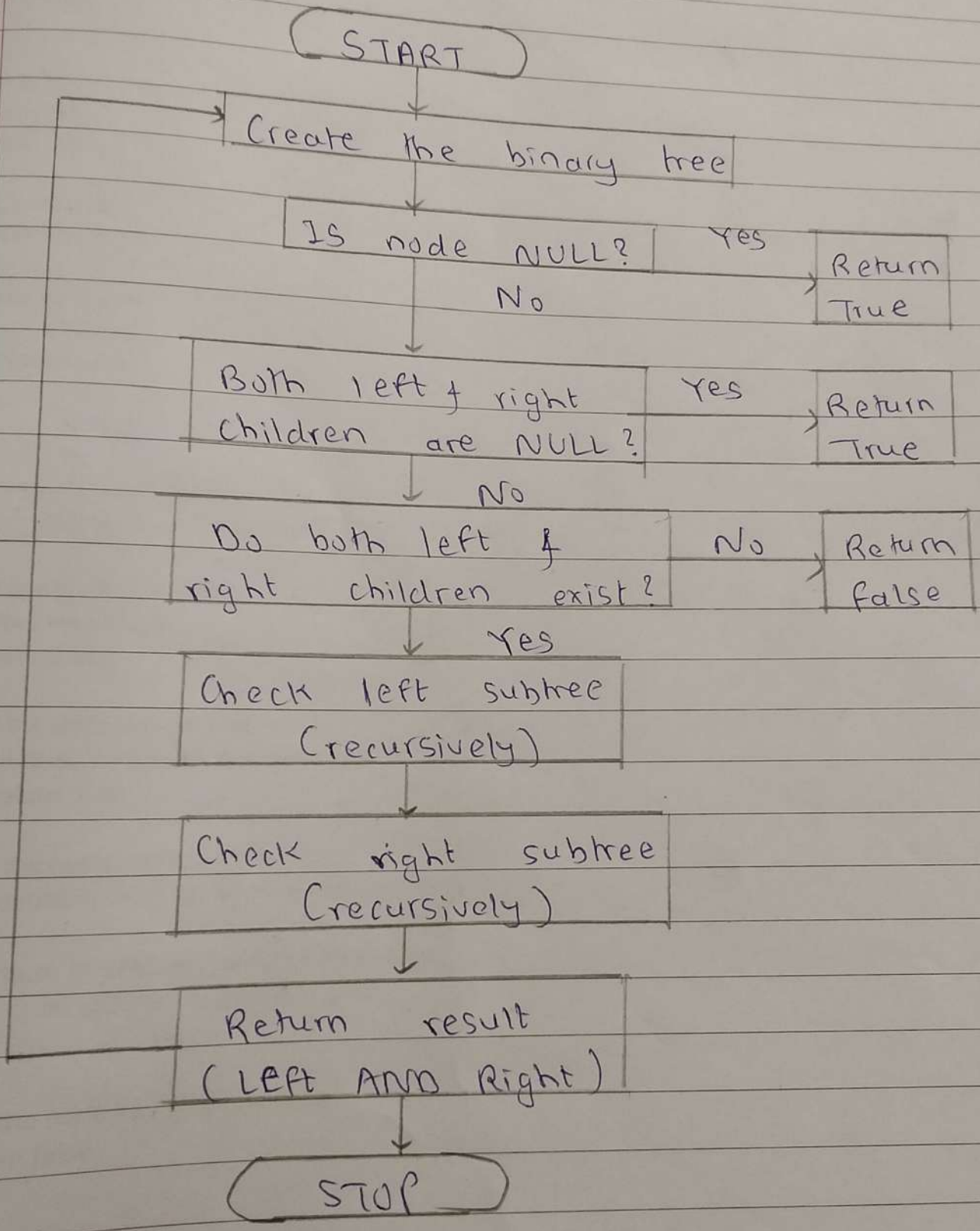
Step 9: In main() func, call isFullBinaryTree() by passing root node.

Step 10: If func. returns true, display that tree is a full binary tree.

Step 11: Otherwise, display that tree is not a full binary tree.

Step 12: Stop the program.

Flowchart :


START
Create the binary tree
Is node NULL?
Yes
Return True
No
Both left & right children are NULL?
Yes
Return True
No
Do both left & right children exist?
No
Return false
Yes
Check left subtree (recursively)
Check right subtree (recursively)
Return result (LEFT AND Right)
STOP

```
#include <iostream>
using namespace std;

// Define a node of the binary tree
class Node
{
public:
    int data;
    Node *left;
    Node *right;

    // Constructor
    Node(int value)
    {
        data = value;
        left = NULL;
        right = NULL;
    }
};

// Function to check full binary tree
bool isFullBinaryTree(Node *root)
{
    // If the node is empty
    if (root == NULL)
        return true;

    // If the node has no children
    if (root->left == NULL && root->right == NULL)
        return true;

    // If the node has two children
    if (root->left != NULL && root->right != NULL)
    {
        return isFullBinaryTree(root->left) &&
               isFullBinaryTree(root->right);
    }

    // If the node has only one child
    return false;
}

int main()
{
    // Create the root node
    Node *root = new Node(1);

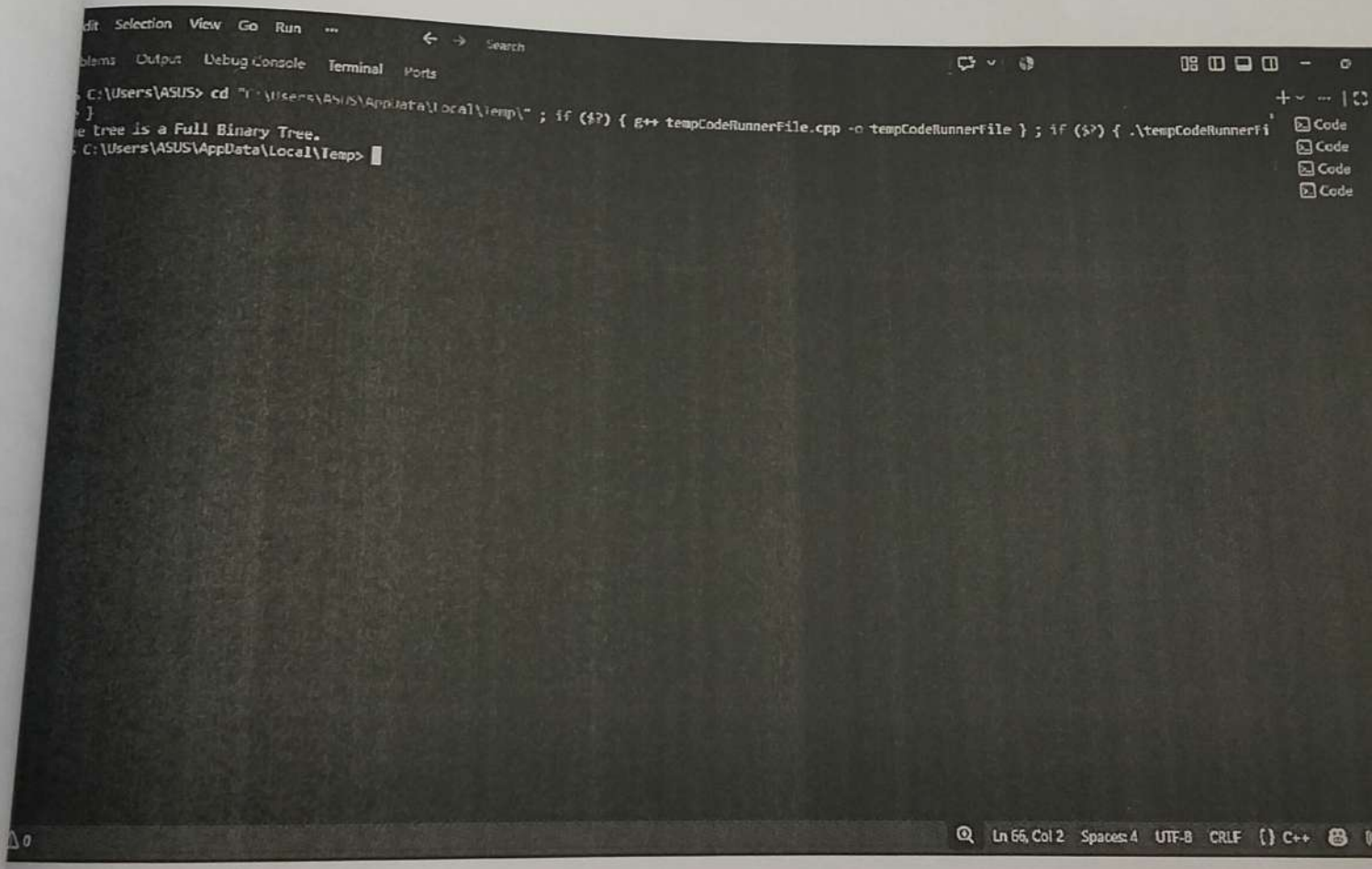
    // Create the child nodes
```

```
    root->left = new Node(2);
    root->right = new Node(3);

    root->left->left = new Node(4);
    root->left->right = new Node(5);

    // Validate the binary tree
    if (isFullBinaryTree(root))
    {
        cout << "The tree is a Full Binary Tree.";
    }
    else
    {
        cout << "The tree is not a Full Binary Tree.";
    }

    return 0;
}
```


dit Selection View Go Run ...
Search
blems Output Debug Console Terminal Ports
C:\Users\ASUS> cd "C:\Users\ASUS\AppData\Local\Temp\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFi
}
e tree is a Full Binary Tree.
C:\Users\ASUS\AppData\Local\Temp>
Code
Code
Code
Code
Ln 66, Col 2 Spaces: 4 UTF-8 CRLF {} C++